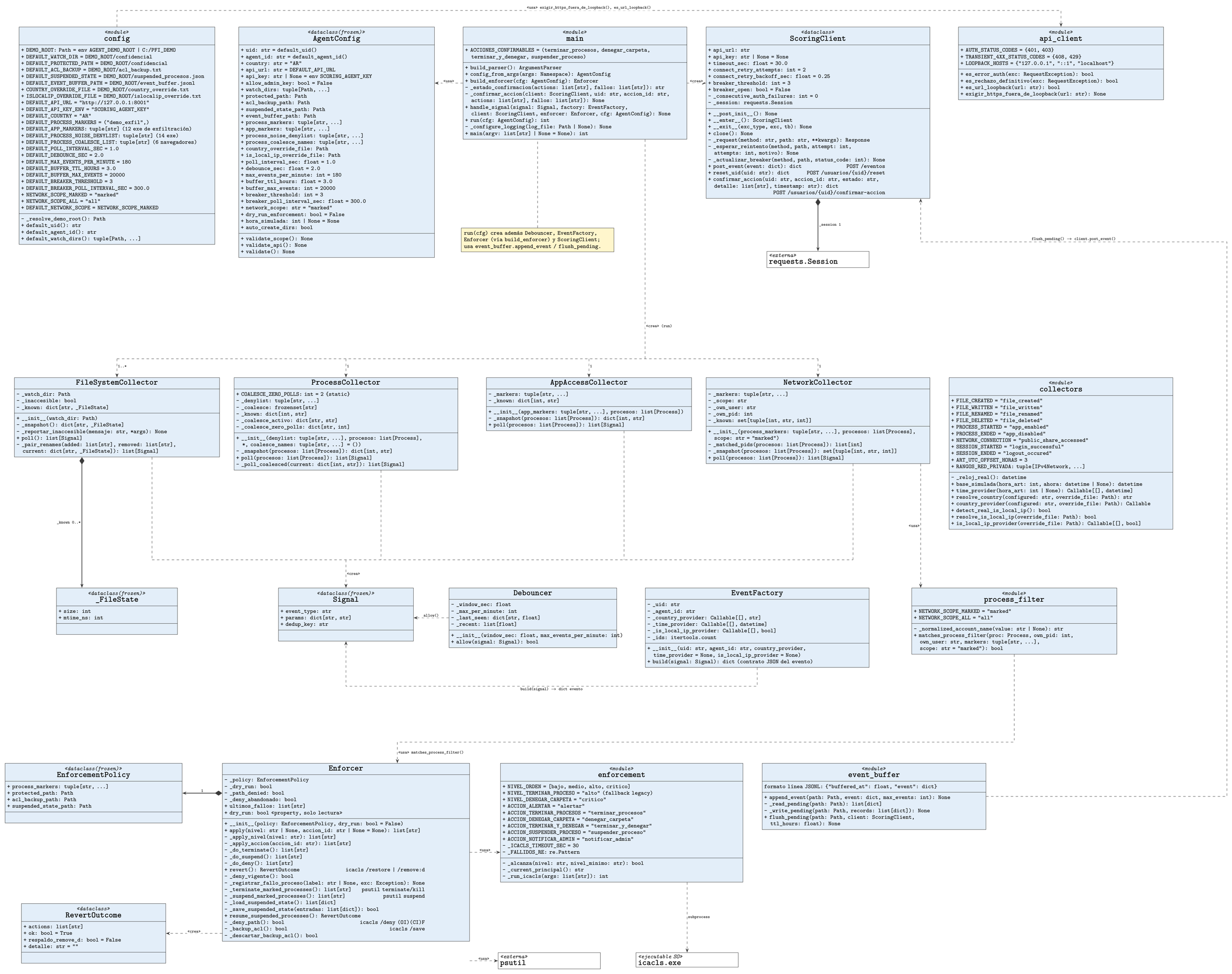

«usa» exigir_https_fuera_de_loopback(), es_url_loopback()
«module»
config
+ DEMO_ROOT: Path = env AGENT_DEMO_ROOT | C:/PFI_DEMO
+ DEFAULT_WATCH_DIR = DEMO_ROOT/confidencial
+ DEFAULT_PROTECTED_PATH = DEMO_ROOT/confidencial
+ DEFAULT_ACL_BACKUP = DEMO_ROOT/acl_backup.txt
+ DEFAULT_SUSPENDED_STATE = DEMO_ROOT/suspended_procesos.json
+ DEFAULT_EVENT_BUFFER_PATH = DEMO_ROOT/event_buffer.jsonl
+ COUNTRY_OVERRIDE_FILE = DEMO_ROOT/country_override.txt
+ ISLOCALIP_OVERRIDE_FILE = DEMO_ROOT/islocalip_override.txt
+ DEFAULT_API_URL = "http://127.0.0.1:8001"
+ DEFAULT_API_KEY_ENV = "SCORING_AGENT_KEY"
+ DEFAULT_COUNTRY = "AR"
+ DEFAULT_PROCESS_MARKERS = ("demo_exfil",)
+ DEFAULT_APP_MARKERS: tuple[str] (12 exe de exfiltración)
+ DEFAULT_PROCESS_NOISE_DENYLIST: tuple[str] (14 exe)
+ DEFAULT_PROCESS_COALESCE_LIST: tuple[str] (6 navegadores)
+ DEFAULT_POLL_INTERVAL_SEC = 1.0
+ DEFAULT_DEBOUNCE_SEC = 2.0
+ DEFAULT_MAX_EVENTS_PER_MINUTE = 180
+ DEFAULT_BUFFER_TTL_HOURS = 3.0
+ DEFAULT_BUFFER_MAX_EVENTS = 20000
+ DEFAULT_BREAKER_THRESHOLD = 3
+ DEFAULT_BREAKER_POLL_INTERVAL_SEC = 300.0
+ NETWORK_SCOPE_MARKED = "marked"
+ NETWORK_SCOPE_ALL = "all"
+ DEFAULT_NETWORK_SCOPE = NETWORK_SCOPE_MARKED
- _resolve_demo_root(): Path
+ default_uid(): str
+ default_agent_id(): str
+ default_watch_dirs(): tuple[Path, ...]
«dataclass(frozen)»
AgentConfig
+ uid: str = default_uid()
+ agent_id: str = default_agent_id()
+ country: str = "AR"
+ api_url: str = DEFAULT_API_URL
+ api_key: str | None = env SCORING_AGENT_KEY
+ allow_admin_key: bool = False
+ watch_dirs: tuple[Path, ...]
+ protected_path: Path
+ acl_backup_path: Path
+ suspended_state_path: Path
+ event_buffer_path: Path
+ process_markers: tuple[str, ...]
+ app_markers: tuple[str, ...]
+ process_noise_denylist: tuple[str, ...]
+ process_coalesce_names: tuple[str, ...]
+ country_override_file: Path
+ is_local_ip_override_file: Path
+ poll_interval_sec: float = 1.0
+ debounce_sec: float = 2.0
+ max_events_per_minute: int = 180
+ buffer_ttl_hours: float = 3.0
+ buffer_max_events: int = 20000
+ breaker_threshold: int = 3
+ breaker_poll_interval_sec: float = 300.0
+ network_scope: str = "marked"
+ dry_run_enforcement: bool = False
+ hora_simulada: int | None = None
+ auto_create_dirs: bool
+ validate_scope(): None
+ validate_api(): None
+ validate(): None
«usa»
«module»
main
+ ACCIONES_CONFIRMABLES = (terminar_procesos, denegar_carpeta, terminar_y_denegar, suspender_proceso)
+ build_parser(): ArgumentParser
+ config_from_args(args: Namespace): AgentConfig
+ build_enforcer(cfg: AgentConfig): Enforcer
- _estado_confirmacion(actions: list[str], fallos: list[str]): str
- _confirmar_accion(client: ScoringClient, uid: str, accion_id: str, actions: list[str], fallos: list[str]): None
+ handle_signal(signal: Signal, factory: EventFactory, client: ScoringClient, enforcer: Enforcer, cfg: AgentConfig): None
+ run(cfg: AgentConfig): int
- _configure_logging(log_file: Path | None): None
+ main(argv: list[str] | None = None): int
run(cfg) crea además Debouncer, EventFactory, Enforcer (vía build_enforcer) y ScoringClient; usa event_buffer.append_event / flush_pending.
«crea»
«dataclass»
ScoringClient
+ api_url: str
+ api_key: str | None = None
+ timeout_sec: float = 30.0
+ connect_retry_attempts: int = 2
+ connect_retry_backoff_sec: float = 0.25
+ breaker_threshold: int = 3
+ breaker_open: bool = False
- _consecutive_auth_failures: int = 0
- _session: requests.Session
+ __post_init__(): None
+ __enter__(): ScoringClient
+ __exit__(exc_type, exc, tb): None
+ close(): None
- _request(method: str, path: str, **kwargs): Response
- _esperar_reintento(method, path, attempt: int, attempts: int, motivo): None
- _actualizar_breaker(method, path, status_code: int): None
+ post_event(event: dict): dict POST /eventos
+ reset_uid(uid: str): dict POST /usuarios/{uid}/reset
+ confirmar_accion(uid: str, accion_id: str, estado: str, detalle: list[str], timestamp: str): dict POST /usuarios/{uid}/confirmar-accion
_session 1
«externa»
requests.Session
«module»
api_client
+ AUTH_STATUS_CODES = {401, 403}
+ TRANSIENT_4XX_STATUS_CODES = {408, 429}
+ LOOPBACK_HOSTS = {"127.0.0.1", "::1", "localhost"}
+ es_error_auth(exc: RequestException): bool
+ es_rechazo_definitivo(exc: RequestException): bool
+ es_url_loopback(url: str): bool
+ exigir_https_fuera_de_loopback(url: str): None
«crea» (run)
flush_pending() → client.post_event()
1..*
1
1
1
FileSystemCollector
- _watch_dir: Path
- _inaccesible: bool
- _known: dict[str, _FileState]
+ __init__(watch_dir: Path)
- _snapshot(): dict[str, _FileState]
- _reportar_inaccesible(mensaje: str, *args): None
+ poll(): list[Signal]
- _pair_renames(added: list[str], removed: list[str], current: dict[str, _FileState]): list[Signal]
ProcessCollector
+ COALESCE_ZERO_POLLS: int = 2 {static}
- _denylist: tuple[str, ...]
- _coalesce: frozenset[str]
- _known: dict[int, str]
- _coalesce_activo: dict[str, str]
- _coalesce_zero_polls: dict[str, int]
+ __init__(denylist: tuple[str, ...], procesos: list[Process], *, coalesce_names: tuple[str, ...] = ())
- _snapshot(procesos: list[Process]): dict[int, str]
+ poll(procesos: list[Process]): list[Signal]
- _poll_coalesced(current: dict[int, str]): list[Signal]
AppAccessCollector
- _markers: tuple[str, ...]
- _known: dict[int, str]
+ __init__(app_markers: tuple[str, ...], procesos: list[Process])
- _snapshot(procesos: list[Process]): dict[int, str]
+ poll(procesos: list[Process]): list[Signal]
NetworkCollector
- _markers: tuple[str, ...]
- _scope: str
- _own_user: str
- _own_pid: int
- _known: set[tuple[int, str, int]]
+ __init__(process_markers: tuple[str, ...], procesos: list[Process], scope: str = "marked")
- _matched_pids(procesos: list[Process]): list[int]
- _snapshot(procesos: list[Process]): set[tuple[int, str, int]]
+ poll(procesos: list[Process]): list[Signal]
«module»
collectors
+ FILE_CREATED = "file_created"
+ FILE_WRITTEN = "file_written"
+ FILE_RENAMED = "file_renamed"
+ FILE_DELETED = "file_deleted"
+ PROCESS_STARTED = "app_enabled"
+ PROCESS_ENDED = "app_disabled"
+ NETWORK_CONNECTION = "public_share_accessed"
+ SESSION_STARTED = "login_successful"
+ SESSION_ENDED = "logout_occured"
+ ART_UTC_OFFSET_HORAS = 3
+ RANGOS_RED_PRIVADA: tuple[IPv4Network, ...]
- _reloj_real(): datetime
+ base_simulada(hora_art: int, ahora: datetime | None): datetime
+ time_provider(hora_art: int | None): Callable[[], datetime]
+ resolve_country(configured: str, override_file: Path): str
+ country_provider(configured: str, override_file: Path): Callable
+ detect_real_is_local_ip(): bool
+ resolve_is_local_ip(override_file: Path): bool
+ is_local_ip_provider(override_file: Path): Callable[[], bool]
_known 0..*
«crea»
«usa»
«dataclass(frozen)»
_FileState
+ size: int
+ mtime_ns: int
«dataclass(frozen)»
Signal
+ event_type: str
+ params: dict[str, str]
+ dedup_key: str
allow()
Debouncer
- _window_sec: float
- _max_per_minute: int
- _last_seen: dict[str, float]
- _recent: list[float]
+ __init__(window_sec: float, max_events_per_minute: int)
+ allow(signal: Signal): bool
EventFactory
- _uid: str
- _agent_id: str
- _country_provider: Callable[[], str]
- _time_provider: Callable[[], datetime]
- _is_local_ip_provider: Callable[[], bool]
- _ids: itertools.count
+ __init__(uid: str, agent_id: str, country_provider, time_provider = None, is_local_ip_provider = None)
+ build(signal: Signal): dict (contrato JSON del evento)
build(signal) → dict evento
«module»
process_filter
+ NETWORK_SCOPE_MARKED = "marked"
+ NETWORK_SCOPE_ALL = "all"
- _normalized_account_name(value: str | None): str
+ matches_process_filter(proc: Process, own_pid: int, own_user: str, markers: tuple[str, ...], scope: str = "marked"): bool
«usa» matches_process_filter()
«dataclass(frozen)»
EnforcementPolicy
+ process_markers: tuple[str, ...]
+ protected_path: Path
+ acl_backup_path: Path
+ suspended_state_path: Path
1
Enforcer
- _policy: EnforcementPolicy
- _dry_run: bool
- _path_denied: bool
- _deny_abandonado: bool
+ ultimos_fallos: list[str]
+ dry_run: bool «property, solo lectura»
+ __init__(policy: EnforcementPolicy, dry_run: bool = False)
+ apply(nivel: str | None, accion_id: str | None = None): list[str]
- _apply_nivel(nivel: str): list[str]
- _apply_accion(accion_id: str): list[str]
- _do_terminate(): list[str]
- _do_suspend(): list[str]
- _do_deny(): list[str]
+ revert(): RevertOutcome icacls /restore | /remove:d
- _deny_vigente(): bool
- _registrar_fallo_proceso(label: str | None, exc: Exception): None
- _terminate_marked_processes(): list[str] psutil terminate/kill
- _suspend_marked_processes(): list[str] psutil suspend
- _load_suspended_state(): list[dict]
- _save_suspended_state(entradas: list[dict]): bool
+ resume_suspended_processes(): RevertOutcome
- _deny_path(): bool icacls /deny (OI)(CI)F
- _backup_acl(): bool icacls /save
- _descartar_backup_acl(): bool
«usa»
«module»
enforcement
+ NIVEL_ORDEN = [bajo, medio, alto, critico]
+ NIVEL_TERMINAR_PROCESO = "alto" (fallback legacy)
+ NIVEL_DENEGAR_CARPETA = "critico"
+ ACCION_ALERTAR = "alertar"
+ ACCION_TERMINAR_PROCESOS = "terminar_procesos"
+ ACCION_DENEGAR_CARPETA = "denegar_carpeta"
+ ACCION_TERMINAR_Y_DENEGAR = "terminar_y_denegar"
+ ACCION_SUSPENDER_PROCESO = "suspender_proceso"
+ ACCION_NOTIFICAR_ADMIN = "notificar_admin"
- _ICACLS_TIMEOUT_SEC = 30
- _FALLIDOS_RE: re.Pattern
- _alcanza(nivel: str, nivel_minimo: str): bool
- _current_principal(): str
- _run_icacls(args: list[str]): int
«module»
event_buffer
formato línea JSONL: {"buffered_at": float, "event": dict}
+ append_event(path: Path, event: dict, max_events: int): None
- _read_pending(path: Path): list[dict]
- _write_pending(path: Path, records: list[dict]): None
+ flush_pending(path: Path, client: ScoringClient, ttl_hours: float): None
«dataclass»
RevertOutcome
+ actions: list[str]
+ ok: bool = True
+ respaldo_remove_d: bool = False
+ detalle: str = ""
«crea»
«usa»
«externa»
psutil
subprocess
«ejecutable SO»
icacls.exe